

ACTIVIDAD 6: Taller Práctico de Repaso y Consolidación en C++

Informe técnico — Normas APA 7.ª edición — Orvion Robotics

Profesor: Mansilla Muñoz York Elías y Yamil Ganduglia | Entrega: 01 de octubre de 2026 |
Versión: v1.0.0 Baseline Aprobado

Integrantes y reparto de tareas

Integrante	Tarea exacta
Malek Cheheid	Informe Apa v7
Teo Laguna	Repositorio y compilación del código
Santiago Osma	Preguntas técnicas

Control de revisiones — DocVer

Versión	Fecha	Cambios	Autor
v0.1.0 Draft	24/9/2026	Borrador inicial cuestionario	Orvion Robotics
v1.0.0 Baseline Aprobado	01/10/2026	Versión final para Moodle + GitHub	Orvion Robotics

Resumen

Este informe documenta la ejecución del taller integrador de repaso (recursividad, búsqueda secuencial y punteros) según el Método 5R: Registrar, Reducir, Responder, Reflexionar y Revisar. Se compiló src/main.cpp en PowerShell con g++ src/main.cpp -o src/repasogeneral.exe y se verificó la salida de los tres retos. El repositorio repasogeneral se protegió con estrategia Dual-Remote (GitHub como principal y GitLab como backup).

```
origin https://github.com/teolaguna/repasogeneral.git (fetch)
origin https://github.com/teolaguna/repasogeneral.git (push)
origin https://gitlab.com/teolaguna/repasogeneral.git (push)
```

link del repositorio: <https://github.com/teolaguna/repasogeneral.git>

1. Método 5R aplicado

R1 Registrar:

```
=====
TALLER INTEGRADOR REPASO - ESTUDIANTE: Teo Laguna
=====

=== RETO 1: SUMA RECURSIVA ===
Suma acumulada de 1 hasta 5: 15

=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===
[SISTEMA] Valor 18 hallado en el índice [6]
          Direccion RAM Hexadecimal: 0x8dcf9ff778

=== RETO 3: INTERCAMBIO DE CELDAS ===
Valores previos  -> X: 100 | Y: 500
Valores actuales -> X: 500 | Y: 100
```

Resultado esperado: Reto 1 = 15; Reto 2 = valor 18 en índice [6] + dirección hexadecimal; Reto 3 = X:100|Y:500 → X:500|Y:100.

R2 Reducir: núcleo 20/80: if (n <= 0), break, temporal.

R4 Reflexionar (PIA/Expotécnica): En nuestro prototipo aplicaríamos los tres pilares así:

1) Búsqueda secuencial (Reto 2): en el loop() ya hacemos if (distancia > 0 && distancia < 25) para detener motores; lo extenderíamos a un buffer int lecturas[10] de distancias y a un array ir[5] de línea, recorriendo celda por celda y con break apenas se halla línea/obstáculo estable, evitando leer todo el arreglo y filtrando picos de ruido del pulseIn.

2) Punteros (Reto 3): usaríamos doble buffer *bufActivo / *bufProceso e intercambio con temporal para no copiar 10 ints en cada ciclo (crítico con 2 KB de SRAM), y pasaje por dirección a gestionarApertura(&estado) y avanzarMotores/detenerMotores.

3) Recursividad (Reto 1): no la usaríamos en el micro para sumar/promediar por riesgo de Stack Overflow; la reemplazamos por versión iterativa con millis() + tiempoEsperaAbrir (6000 ms) como ya hace el código para abrir la tapa a 90° tras 6 s.

R5 Revisar: sin errores de compilación, git push a ambos remotos y actualización de

docs/CHANGELOG.md y carpeta de campo.

2. Cuestionario técnico de control

1. Caso Base: Si una función recursiva no tiene caso base de corte, genera recursión infinita y desbordamiento de pila (Stack Overflow). Cada llamada apila en la RAM un marco con dirección de retorno, parámetros y variables locales sin liberar los anteriores. Al agotarse el Stack asignado, el programa se congela o aborta por falla de segmentación. En el taller el `if (n <= 0) return 0;` evita eso y permite que `sumaRecursiva(5)` se desenrolle como $5+4+3+2+1=15$.

2. Eficiencia: `break` evita trabajo inútil de CPU. La búsqueda secuencial es $O(n)$. Sin `break` recorre las 10 celdas aunque el 18 esté en el índice [6]. Con `break` se interrumpe ahí y no accede a [7-9], ahorrando iteraciones, accesos a RAM y ciclos de reloj. Pasa de siempre 10 comparaciones a solo 7 en este caso.

3. Direcccionamiento: Con `&vectorDatos[i]` se imprime la dirección hexadecimal real de cada celda. Al ser int de 4 bytes, debe cumplirse `&arr[i+1]-&arr[i]==4`, ej. 0x1000, 0x1004, 0x1008... Eso prueba que el array es un bloque contiguo. En la ejecución `&vectorDatos[6]` muestra dónde está el 18, a 24 bytes de la base.

4. Variable temporal: Guarda el valor original antes de sobrescribirlo. Con `temporal=*ptrA(100); *ptrA=*ptrB(500); *ptrB=temporal(100);` se logra X:500|Y:100. Sin temporal, al hacer `*ptrA=*ptrB` se perdería el 100 y quedaría 500/500. Junto a `&x,&y` y `*ptrA,*ptrB` permite modificar las variables originales fuera de la función.

5. Autoevaluación: El Reto 1 fue el más complejo por no ver la pila. Lo resolvimos trazando $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$ a mano y quitando el `if` en prueba para provocar el desborde y entender el caso base.